Ders 5 Genişletilmiş Notu: NFA'dan DFA'ya Dönüşümün Tamamlanması, Pompalama Önsavı ve Düzenli Olmayan Dillerin İspatı

Ders 5 Genişletilmiş Notu: NFA'dan DFA'ya Dönüşümün Tamamlanması, Pompalama Önsavı ve Düzenli Olmayan Dillerin İspatı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. NFA'dan DFA'ya Dönüşüm Örneğinin Tamamlanması
- 2. Dönüşümün Genel Yöntemi
- 3. Düzenli Olmayan Diller
- 4. Güvercin Yuvası İlkesi
- 5. Sonlu Otomatta Tekrar Eden Durum
- 6. Dizgiyi Üç Parçaya Bölmek: x, y, z
- 7. Pompalama Önsavı
- 8. Pompalama Önsavıyla Düzensizlik İspatı
- 9. Örnek 1: $\{a^n b^n \mid n \geq 0\}$
- 10. Örnek 2: $\{a^n b^l c^{n+l} \mid n, l \geq 0\}$
- 11. Örnek 3: $\{a^{n!} \mid n \geq 0\}$
- 12. Özet: Düzenlilik ve Düzensizlik Nasıl İspatlanır?

1. NFA'dan DFA'ya Dönüşüm Örneğinin Tamamlanması

1.1 Geçen haftanın örneği ve bir düzeltme

Geçen hafta deterministik olmayan sonlu otomatların (non-deterministic finite automata, NFA) deterministik sonlu otomatlara (deterministic finite automata, DFA) dönüştürülmesine başlamıştık. Bu derse aynı örnekle devam ediyoruz. NFA'nın geçişleri şunlardır:

$$q_0 \xrightarrow{a} q_1, \qquad q_1 \xrightarrow{a} q_1, \qquad q_1 \xrightarrow{\varepsilon} q_2, \qquad q_2 \xrightarrow{b} q_0$$

Başlangıç durumu q_0'dır. Hoca, geçen hafta çizilen NFA'da bir eksik olduğunu söyledi: q_1 bir kabul durumudur (accept state). Geçen hafta bu işaretlenmemişti, bu yüzden notlarınızda q_1'i kabul durumu yapın.

Amacımız bu makineyi DFA'ya çevirmektir. Elde edeceğimiz makinede ε geçişi, yani boş geçiş (null transition) olmayacaktır. Ayrıca alfabe açıkça $\{a, b\}$ olduğu için her durumda hem a hem b için ne olacağını söylemek zorundayız.

1.2 Başlangıç durumu ve durumların adlandırılması

DFA'nın durumlarına aslında istediğimiz adı verebiliriz. Ancak NFA'dan DFA'ya dönüştürürken durumları **kümelerle** adlandırırız. Bir küme adı, o DFA durumunun NFA'nın hangi durumlarını

temsil ettiğini söyler. DFA'nın başlangıç durumu, tek elemanı q_0 olan kümedir: $\{q_0\}$.

Bundan sonra yöntem tek bir sorudan ibarettir. Her durum için şunu sorarız: Bu durum a alınca ne olur, b alınca ne olur?

> **Hocanın vurgusu:** Bu soruyu tekrar tekrar sormanız gerekir. Yeni durum oluştuğu sürece, oluşan her yeni durum için aynı soru sorulur. Yeni durum çıkmadığında iş biter.

1.3 $\{q_0\}$ durumu

Verilen: $\{q_0\}$ durumundayız.

a **gelirse.** q_0, a ile q_1'e gider. Ama q_1'den çıkan bir boş geçiş vardır, bu yüzden makine hemen q_2'de de olur. Yani aynı anda hem q_1'de hem q_2'deyizdir. Dolayısıyla hedef DFA durumu $\{q_1, q_2\}$'dir. Bu durum, özgün NFA'daki q_1 ve q_2'yi birlikte temsil eder.

b **gelirse.** q_0'dan çıkan bir b geçişi yoktur. NFA'da bu, o yolun kabul etmediği anlamına gelir. Ama DFA'da b bir yere gitmek zorundadır. Bu yüzden yalnızca bunun için bir durum oluştururuz ve ona boş durum (empty state) deriz: $\emptyset$. Bu bir ret durumudur (rejecting state), b ile ona gitmekte de bir sorun yoktur. Birazdan göreceğimiz gibi, bu durumdan dışarı çıkılmasına izin vermeyeceğiz.

Elimizde artık iki yeni durum var: $\{q_1, q_2\}$ ve $\emptyset$. Aynı soruyu ikisi için de soracağız.

1.4 $\{q_1, q_2\}$ durumu

a **gelirse.** q_1, a ile kendisine gider. q_2'nin ise çıkan a geçişi yoktur, ondan hiçbir şey gelmez. Yani a ile q_1'e gideriz. Hoca burada "q_1 zaten bu durumun içinde, bu durum özgün q_1 ve q_2'nin ikisini de temsil ediyor" diyerek geçişi döngüye çevirdi: $\{q_1, q_2\} \xrightarrow{a} \{q_1, q_2\}$.

> **Ders sonrası not:** Döngünün asıl gerekçesi boş geçiştir. a ile yalnızca q_1'e gidilir, sonra $q_1 \xrightarrow{\varepsilon} q_2$ geçişi hedefe q_2'yi de ekler. Bu yüzden sonuç $\{q_1\}$ değil $\{q_1, q_2\}$ olur. q_1'in "zaten kümede olması" tek başına gerekçe değildir.

b **gelirse.** q_1'in b geçişi yoktur. q_2'nin vardır: $q_2 \xrightarrow{b} q_0$. Dolayısıyla $\{q_1, q_2\} \xrightarrow{b} \{q_0\}$ olur. $\{q_0\}$ durumunu zaten oluşturmuştuk, yeni durum çıkmaz.

1.5 $\emptyset$ durumu: tuzak durum

Hoca bu duruma öldürme durumu, ölü durum (death state) dedi. Hem a hem b için bu durumda kalırız:

$$\emptyset \xrightarrow{a} \emptyset, \qquad \emptyset \xrightarrow{b} \emptyset$$

Giriş b ile başlarsa bu duruma geliriz. Ondan sonra ne gelirse gelsin burada kalırız, çıkamayız. Durum ret durumu olduğu için dizgi reddedilir. Bu davranış özgün NFA ile uyumludur: NFA'da kabul durumuna gitmek için a ile başlamak gerekir, b ile başlayan dizgi kabul edilmez.

1.6 Kabul durumları

Bir adım daha kaldı: DFA'da henüz kabul durumu yok. NFA'da q_1 kabul durumudur. Bu yüzden içinde q_1 bulunan her DFA durumunu kabul durumu yaparız. Burada bu durum $\{q_1, q_2\}$'dir. Çizimde etrafına ikinci bir daire çizilir.

Sonuç DFA:

DFA durumu	a	b	Kabul?
$\{q_0\}$ (başlangıç)	$\{q_1, q_2\}$	$\emptyset$	Hayır
$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_0\}$	Evet (q_1 içeriyor)

DFA durumu	a	b	Kabul?
$\emptyset$ (tuzak)	$\emptyset$	$\emptyset$	Hayır

Bu tablo alt küme yapısıyla yeniden hesaplandı. NFA ile DFA, 14 sembole kadar bütün dizgilerde (32767 dizgi) aynı kararı verdi.

2. Dönüşümün Genel Yöntemi

Hoca örnekten sonra yöntemi genel olarak yazdı, ancak örneğin daha açıklayıcı olduğunu da ekledi. Yöntem şu adımlardan oluşur.

1. **DFA'nın durumlarını belirle.** NFA'nın durumları $q_0, q_1, \ldots$ ise DFA'nın durumları $\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}, \ldots$ olur. Bu, NFA durum kümesi Q'nun kuvvet kümesidir (power set).
2. **Başlangıç durumunu yaz.** NFA'nın başlangıç durumu q_0 ise DFA'nınki $\{q_0\}$'dır.
3. **Geçişleri hesapla.** Her DFA durumu $\{q_i, q_j, \ldots, q_m\}$ ve alfabedeki her σ sembolü için önce q_i'nin NFA'da σ ile gittiği durumlar bulunur ve sonuç kümesine eklenir. Aynısı q_j için yapılır ve bu, kümedeki son duruma kadar sürer. Yapılan işlem bir birleşimdir (union).
4. **Yeni durumu tabloya yaz.** Elde edilen $\{q'_k, q'_l, \ldots\}$ kümesi yeni DFA durumudur. Geçiş tablosuna "$\{q_i, q_j, \ldots, q_m\}$ durumunda σ gelince $\{q'_k, q'_l, \ldots\}$ durumuna gidilir" diye yazılır.
5. **Tekrarla.** Yeni durum eklenmeyene kadar 3. ve 4. adımlar sürdürülür. O noktada bütün durumlar bulunmuştur.
6. **Kabul durumlarını yaz.** NFA'nın her kabul durumu q_f için, içinde q_f bulunan bütün DFA durumları kabul durumu yapılır.

   Hocanın vurgusu: Son adımı, yani kabul durumlarını yazmayı unutmayın.

   Ders sonrası not: Hocanın genel yazımında boş geçişler yer almıyor, oysa örnekte kullanıldılar. NFA'da ε geçişi varsa iki yerde boş geçişle ulaşılan durumlar da kümeye eklenmelidir. Başlangıç durumu $\{q_0\}$ ile birlikte q_0'dan boş geçişlerle ulaşılan durumların kümesidir. Her birleşimden sonra da sonuç kümesinden boş geçişlerle ulaşılan durumlar eklenir. Bu örnekte q_0'dan boş geçiş olmadığı için başlangıç durumu $\{q_0\}$ kalır, ama q_1'e her gelişte q_2 kümeye girer.

   Ek bilgi (derste anlatılmadı): $|Q|$ durumlu bir NFA'nın kuvvet kümesinde $2^{|Q|}$ alt küme vardır, ama bunların hepsini yazmak gerekmez. Başlangıçtan ulaşılabilen kümeler yeterlidir ve "yeni durum çıkmayana kadar" kuralı tam olarak bunları üretir. Bu örnekte 3 NFA durumu 8 alt küme verir, tabloda ise yalnız 3 tanesi çıkar. Bu DFA'nın dili $a(a \cup ba)^*$ düzenli ifadesiyle yazılabilir. Bu eşitlik 14 sembole kadar bütün dizgilerde hesapla denendi.

3. Düzenli Olmayan Diller

Dönüşümü bitirdikten sonra pompalama önsavına (pumping lemma) geçiyoruz. Bu önsav, düzenli olmayan dilleri bulmamıza yardım eder.

Şimdiye kadar gördüklerimizi hatırlayalım: Kendisini kabul eden bir NFA ya da DFA bulabildiğimiz dillerin kümesine düzenli diller (regular languages) denir. a^*b ya da $b^*c + a$ gibi ifadelerle yazılan diller bu türdendir. Bunlar için bir sonlu otomat (finite automaton) tasarlayabiliriz. "Düzenli" sözcüğünün anlamı budur: Bu dil için bir sonlu otomat bulabiliriz.

Ama bazı diller böyle değildir. Hocanın verdiği iki örnek şunlardır:

$$\{a^n b^n \mid n \geq 0\}, \qquad \{ww^R \mid w \in \{a, b\}^*\}$$

İkinci dil, bir dizgiden ve onun tersinden (reverse) oluşan dizgilerdir. Bu dillere düzenli olmayan (non-regular, irregular) diller denir. Bunlar için kabul eden bir sonlu otomat bulamayız ya da tasarlayamayız.

Peki bunu nasıl ispatlayacağız? Bir L dilinin düzenli olmadığını göstermek için pompalama önsavı denen gözlemi kullanacağız. Pompalama önsavı da güvercin yuvası ilkesinin (pigeonhole principle) bir uygulamasıdır.

4. Güvercin Yuvası İlkesi

Bu ilke çok basittir. n tane güvercininiz ve m tane güvercin yuvanız olsun. $n > m$ olsun, yani güvercinler yuvalardan fazla olsun. Güvercinleri bu yuvalara yerleştirirseniz, içinde birden fazla güvercin bulunan en az bir yuva olur.

Pompalama önsavında yuvaların makinenin durumları, güvercinlerin ise makinenin bir dizgiyi okurken geçtiği durumlar olduğunu birazdan göreceğiz.

5. Sonlu Otomatta Tekrar Eden Durum

5.1 Dört durumlu örnek makine

Dört durumlu bir sonlu otomata bakalım. Durumlar q_1, q_2, q_3, q_4'tür, başlangıç q_1, kabul durumu q_4'tür. Geçişler şunlardır:

$$q_1 \xrightarrow{a} q_2, \quad q_2 \xrightarrow{b} q_2, \quad q_2 \xrightarrow{a} q_3, \quad q_3 \xrightarrow{a} q_2, \quad q_3 \xrightarrow{b} q_4, \quad q_4 \xrightarrow{b} q_4, \quad q_4 \xrightarrow{a} q_3$$

> **Ders sonrası not:** Hoca bu makineye DFA dedi. Yukarıdaki listede q_1'in b geçişi yoktur. Bölüm 1.3'teki kurala göre tam bir DFA'da $q_1 \xrightarrow{b}$ için de bir hedef, örneğin bir tuzak durum gerekir. Aşağıdaki akıl yürütme bundan etkilenmez, çünkü kullanılan bütün dizgiler a ile başlar.

5.2 Yürüyüş

Bir dizgi verildiğinde makinenin geçtiği durumların sırasına yürüyüş (walk) diyeceğiz.

$w = aabb$ **(4 sembol).** q_1'den başlarız, çünkü başlangıç durumu odur. a ile q_2'ye, ikinci a ile q_3'e, b ile q_4'e gideriz. Üç sembol sonra bütün durumları ziyaret etmiş olduk. Sıradaki b için bu durumlardan biri mutlaka tekrar edecek. Gerçekten de b ile yine q_4'te kalırız:

$$q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_3 \xrightarrow{b} q_4 \xrightarrow{b} q_4$$

Burada q_4 tekrar etmiştir.

$w = aaab$ **(kabul edilmeyen deneme).** Hoca önce bunu denedi: q_1, q_2, q_3, sonra üçüncü a ile q_2'ye geri döneriz. Son b ile kabule ulaşamayız, dizgi kabul edilmez. Hoca kabul edilen bir örnek istediği için bir a daha ekledi.

$w = aaaab$ **(5 sembol).** Dört a ve bir b vardır:

$$q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_3 \xrightarrow{a} q_2 \xrightarrow{a} q_3 \xrightarrow{b} q_4$$

Son durum q_4 kabul durumudur, dizgi kabul edilir. Makinede dört durum, dizgide beş sembol vardır. Yürüyüşte hem q_3 hem q_2 tekrar eder. Üstelik yalnız tekrar yoktur, bir döngü (loop) de vardır. Hoca $aabb$ yürüyüşündeki b döngüsünü de gösterdi ve makinede birden çok döngü olduğunu söyledi.

Buradan çıkan sonuç şudur. Sembol sayısı en az durum sayısı kadar olan bir dizginin yürüyüşünde en az bir durum tekrar eder ve orada bir döngü oluşur.

5.3 Güvercin yuvası ilkesi burada nasıl uygulanır?

Makinede yalnızca q_1, q_2, q_3, q_4 durumları vardır. Bunlar güvercin yuvalarıdır. Yürüyüşteki durumlar ise güvercinlerdir.

- $aaaab$ yürüyüşünde 6 güvercin ($q_1, q_2, q_3, q_2, q_3, q_4$), 4 yuva vardır.
- 4 sembollü bir dizgide başlangıç durumu da sayıldığı için 5 güvercin ($aabb$ için q_1, q_2, q_3, q_4, q_4), 4 yuva vardır.

Güvercinleri yuvalara eşlemeye çalıştığımızda en az iki güvercin aynı yuvaya düşer. Bu, bir durumun tekrar etmesi demektir.

Bu ilk gözlemin genel hali şudur: Dizginin uzunluğu durum kümesinin büyüklüğünden küçük değilse, yani $|w| \geq |Q|$ ise, güvercin yuvası ilkesi gereği w'nin yürüyüşünde bir durum tekrar eder.

6. Dizgiyi Üç Parçaya Bölmek: x, y, z

Durumlar arasında ilerlerken bir noktada bir döngüye gireriz. Bu resim, kabul edilen dizgiyi üç parçaya bölmemizi sağlar. Bu parçalara x, y ve z diyelim. Hoca özellikle uyardı: x, y, z durumlar değildir, **dizginin parçalarıdır**.

6.1 Birinci gözlem: y tekrarlanabilir ya da çıkarılabilir

Kabul edilen w dizgisi $w = xyz$ biçiminde yazılabilir. Burada y tekrarlanabilir ya da çıkarılabilir, elde edilen dizgi yine kabul edilir.

$aaaab$ örneğine dönelim. Yürüyüş $q_1, q_2, q_3, q_2, q_3, q_4$ idi ve q_2, q_3 bölümü tekrar ediyordu. Bu tekrarlardan birini çıkarırsak makine yine kabul eder. İstersek yeni q_2, q_3 de ekleyebiliriz, örneğin $q_1, q_2, q_3, q_2, q_3, q_2, q_3$ ve sonunda q_4. İstediğimiz kadar ekleyebiliriz, sonlu otomat sonunda q_4'e gidip kabul eder.

Tekrar eden durumlar döngü kısmına karşılık gelir. Döngüden istediğimiz kadar kullanabileceğimiz için parçalara şu adları veririz:

- x: döngüden önceki kısım. Örnekte $q_1 \to q_2$ geçişi, yani $x = a$.
- y: tekrar eden kısım. Örnekte $q_2 \to q_3 \to q_2$, yani $y = aa$.
- z: döngüden sonraki kısım. Örnekte $q_2 \to q_3 \to q_4$, yani $z = ab$.

Bu bölmeyle y'yi hiç kullanmazsak aab, iki kez kullanırsak $aaaaaab$ elde ederiz. İkisi de kabul edilir. y'nin 0'dan 7'ye kadar bütün tekrar sayıları simülasyonla denendi, hepsi kabul edildi.

6.2 İkinci gözlem: $|xy| \leq m$

m makinedeki durum sayısı olsun. İkinci gözlem, x ile y'nin birlikte uzunluğunun en fazla m olduğudur:

$$|xy| \leq m$$

$aaaab$ örneğinde $m = 4$'tür ve $x = a$, $y = aa$ olduğu için $|xy| = 3 \leq 4$ olur.

> **Ders sonrası not:** Hoca bu örnekte $|xy|$'yi "iki" diye saydı, çünkü q_2, q_3 durumlarını saydı ve x'i katmadı. Doğrusu $|xy| = 3$'tür. Hoca ayrıca burada "z yok" dedi. Bu örnekte $z = ab$'dir.

Nedenini görmek için genel resme bakalım. Bir dizi durumdan geçip bir noktada döngüye giriyoruz. Hocanın çizimindeki adlarla $\sigma_1, \sigma_2, \sigma_3$ döngüden önce okunan semboller olsun, yani $x = \sigma_1\sigma_2\sigma_3$. $\sigma_4, \sigma_5, \sigma_6$ de döngüde okunan semboller olsun, yani $y = \sigma_4\sigma_5\sigma_6$. Döngünün başladığı duruma q diyelim.

Bu yolda q dışında hiçbir durumu yeniden kullanmadık. q dışındaki bütün durumlar yalnız bir kez kullanılıyor. Makinede m durum olduğuna göre x ile y'nin uzunluğu en fazla m olabilir. Eşitlik, q'nun tekrar ettiği durumda bütün durumların kullanılmasıyla olur ve q yalnız bir kez tekrar edebilir.

Ders sonrası not: "q dışında hiçbir durum yeniden kullanılmadı" cümlesi, q yürüyüşte tekrar eden **ilk** durum olarak seçilirse doğrudur. Uzun bir yürüyüşte başka tekrarlar da olabilir. İlk tekrara kadar en fazla $m+1$ durum sayılır, bu da en fazla m sembol demektir. Dört durumlu makinede, 4 ile 14 uzunluk arasındaki kabul edilen 5453 dizginin hepsinde bu seçim $|xy| \leq 4$, $|y| \geq 1$ koşullarını sağladı ve pompalama denemesi başarılı oldu.

6.3 Üçüncü gözlem: $|y| \geq 1$

y bir döngü olduğuna göre içinde en az bir geçiş bulunmalıdır. Kendine dönen tek bir geçişten oluşan en basit döngüde bile bir geçiş vardır. Bu yüzden y'nin uzunluğu en az 1'dir:

$$|y| \geq 1$$

7. Pompalama Önsavı

7.1 Birinci gözlemi i ile yazmak

Aradan sonra hoca birinci gözlemi biraz daha işledi. Durumlar ve geçişler var, bir noktada bir döngü var. Döngüden önceki geçişlere x, döngüden sonrakilere z, döngünün içindekilere y diyoruz.

$w = xyz$ zaten kabul edilir, bunu biliyoruz. Şimdi $xyyz$ dizgisine bakalım. Bu da kabul edilir. Çünkü x bizi döngünün başına getirir. İlk y yine döngünün başına getirir. Sonraki y de yine döngünün başına getirir. Son olarak z bizi kabul durumuna götürür.

Hatta istediğimiz sayıda y kullanabiliriz. Bunu kısaca şöyle yazarız:

$$xy^i z \text{ kabul edilir}, \qquad i = 0, 1, 2, ...$$

İlginç olan, $i = 0$'ın da kabul edilmesidir. y^0 demek y yok demektir, yani dizgi xz olur. Yürüyüşe baktığımızda x'ten sonra döngüye hiç girmeden doğrudan z ile devam ederiz ve yine kabul durumuna varırız. "Kabul edilir" yerine "makinenin dilindedir" de diyebiliriz: $xz \in L(M)$.

7.2 Önsavın ifadesi

Bütün bu gözlemler birleşince pompalama önsavını elde ederiz.

> **Pompalama önsavı.** L sonsuz bir düzenli dil olsun. Bir m tamsayısı vardır (kritik uzunluk, critical length) öyle ki, L'deki uzunluğu en az m olan her w dizgisi $w = xyz$ biçiminde yazılabilir ve
>
> - $|xy| \leq m$,
> - $|y| \geq 1$,
> - her $i \geq 0$ için $xy^i z \in L$.
>
> **Hocanın vurgusu:** Burada "düzenli" sözcüğü anahtardır. Önsav düzenli diller için söylenmiştir.

Sipser'ın kitabında m yerine p kullanılır ve buna pompalama uzunluğu (pumping length) denir. Yani "kritik uzunluk" ya da "pompalama uzunluğu" diyebilirsiniz, gösterim de m ya da p olabilir.

> **Ders sonrası not:** Hoca önsavı sonsuz düzenli diller için yazdı. Önsav bütün düzenli diller için geçerlidir: Sonlu bir dilde m en uzun dizgiden büyük seçilirse uzunluğu en az m olan dizgi kalmaz ve koşul kendiliğinden sağlanır. Ayrıca altyazıda "dil tarafından kabul edilen dizgi" deniyor. Dizgiyi makine kabul eder, dizgi ise dilin içindedir.

8. Pompalama Önsavıyla Düzensizlik İspatı

8.1 Sonlu diller düzenlidir

Önsavı ne zaman kullanırız? Bir dilin düzenli **olmadığını** kanıtlamaya çalışırken. Önce şunu görelim: Bir dil boyut olarak sonluysa düzenlidir. Sonlu sayıda dizgiden oluşan bir dil için, onu kabul eden bir sonlu otomatı kolayca kurabiliriz.

Bu nedenle düzenli olmayan her dil sonsuzdur. Demek ki pompalama önsavını yalnız sonsuz bir dilin düzenli olmadığını göstermek için kullanırız. Dil sonluysa sorun yoktur, düzenlidir. Sonsuzsa önsavı kullanmayı deneyebiliriz.

> **Ek bilgi (derste anlatılmadı):** Önsav tek yönlüdür: "L düzenliyse önsavın koşulları sağlanır" der. Tersi doğru değildir, koşulları sağlayan ama düzenli olmayan diller vardır. Bu yüzden önsav düzenliliği kanıtlamak için kullanılamaz. Yalnız koşulların bozulduğunu göstererek düzensizlik kanıtlanır.

8.2 İspatın yapısı

İspat her zaman **çelişkiyle ispattır** (proof by contradiction). Sonsuz bir L dilinin düzenli olmadığını göstermek için L'nin düzenli olduğunu varsayarız. O zaman pompalama önsavı L için geçerli olmalıdır. Önsavı kullanarak bir çelişki elde ederiz. Sonuç: L düzenli değildir.

Adımlar ayrıntılı olarak şöyledir. Hoca 0. adımı ilk örneği yazarken ekledi.

0. Çelişki için L'nin düzenli olduğunu varsay. L sonsuz olduğundan pompalama önsavı geçerli olmalıdır.
1. m kritik uzunluk olsun. m bir sayı değildir, m olarak kalır.
2. L'de, uzunluğu en az m olan bir w dizgi deseni (string pattern) seç. Desende m geçmelidir.
3. $w = xyz$ yaz.
4. y'lerin sayısını değiştirerek elde edilen $w' = xy^iz$ dizgisinin, $i \neq 1$ olan bir i için dilde olmadığını göster. $i = 1$ için dizgi zaten dildedir, bu yüzden başka bir i gerekir. Bu, aradığımız çelişkidir.

   Hocanın vurgusu: İkinci adım önemlidir. Seçtiğiniz desende m bulunmalı ve desenin uzunluğu en az m olmalıdır.

Çelişki için **tek bir** dizgi deseninin çelişki vermesi yeterlidir. Ama seçilen desenin **bütün** olası xyz bölmeleri için çelişki gösterilmelidir. Hoca bunu ilk iki örnekte açıkça söyledi. $|xy| \leq m$ ve $|y| \geq 1$ koşulları olası bölmelerin sayısını azalttığı için bu iş sandığımızdan kolaydır.

Hoca bu anlatımın soyut olduğunu, örneklerle anlaşılacağını söyledi. Üç örnek yapıldı.

9. Örnek 1: $\{a^n b^n \mid n \geq 0\}$

Teorem. $L = \{a^n b^n \mid n \geq 0\}$ düzenli değildir.

Önce dile bakalım. $n = 0$ için boş dizgi, $n = 1$ için ab, $n = 2$ için $aabb$ elde ederiz. Yani önce bir miktar a, ardından aynı sayıda b gelir.

Adım 0. Çelişki için L'nin düzenli olduğunu varsayalım. L sonsuz olduğundan pompalama önsavı geçerli olmalıdır.

Adım 1. m kritik uzunluk olsun.

Adım 2. İçinde m geçen bir desen arıyoruz. Çok doğal bir seçim $w = a^m b^m$'dir, çünkü dilin kendisi de $a^n b^n$ biçimindedir. Uzunluğu $|w| = 2m$'dir ve $2m \geq m$ olduğundan koşul sağlanır.

Adım 3. Bölmeyi inceleyelim. Dizgi m tane a ve ardından m tane b'den oluşur:

$$w = \underbrace{a \cdots a}_{m} \underbrace{b \cdots b}_{m}$$

Burada $|xy| \leq m$ koşulu çok yardımcı olur. x dizginin başından başlar ve xy en fazla m uzunluktadır. Bu yüzden y'nin b'lere uzandığı bir bölme yapılamaz. Hoca böyle bir bölmeyi çizip "bunu yapabilir miyiz? Hayır, çünkü $|xy|$ en fazla m olmalı" dedi.

Öyleyse x ve y yalnız a'lardan oluşur. y'de k tane a olsun:

$$y = a^k, \qquad 1 \leq k \leq m$$

k en az 1'dir, çünkü y'de en az bir eleman bulunmalıdır. En fazla m'dir, çünkü x boş olup ilk m sembolün tamamı y olabilir. x'in tam olarak ne olduğu önemli değildir.

> **Hocanın vurgusu:** En büyük kolaylık, x ile y'nin birlikte en fazla m uzunlukta olması ve dizginin başında bulunmasıdır. Bu, olası bölmeleri daraltır.

Adım 4. $i = 2$ alalım, yani y'lerin sayısını birden ikiye çıkaralım. Bir a^k daha gelir:

$$xy^2z = a^{m+k}b^m$$

Önsava göre bu dizgi dilde olmalıdır. Ama bu dilde a sayısı ile b sayısı eşit olmak zorundadır. $k \geq 1$ olduğu için $m + k \neq m$'dir, dizgi L'de değildir. Bu bir çelişkidir.

Sonuç. L'nin düzenli olduğu varsayımı yanlıştır. L düzenli bir dil değildir.

> **Ders sonrası not:** Hoca burada iki kez sürçtü. Önce "$k = 2$" dedi, sonra "y'lerin sayısını birden ikiye çıkarıyoruz" diye kendini düzeltti; söz konusu olan $i = 2$'dir. Ardından çelişkiyi "k birden büyükse" diye yazdı ve "$k = 1$ ise sorun yok" dedi. Doğrusu, her $1 \leq k \leq m$ için çelişki vardır, $k = 1$ de $a^{m+1}b^m \notin L$ verir. Tahtadaki "$k > 1$" yazıları $k \geq 1$ diye okunmalıdır.

Soru: Burada i neyi, k neyi temsil ediyor? **Cevap:** Dizgiyi xy^iz biçiminde yazıyoruz ve y'nin sayısını değiştirdiğimizde dizgilerin hâlâ dilde olması gerektiğini söylüyoruz. Demek ki i, y'nin kaç kez tekrar edildiğidir. Burada y olarak a^k seçtik, yani k da y'nin içindeki a sayısıdır. Özgün dizgi xa^kz idi ve bu a^mb^m'ye eşittir. i'yi 2 yaparsak xa^ka^kz olur. Fazladan bir a^k gelmiştir ve bunu a'ların yanına yazarız. Bu yüzden üs $m + k$ olur.

Hoca aynı yanıtta genel biçimi de verdi. i'den yola çıkarsak $i - 1$ tane fazladan a^k gelir:

$$xy^iz = a^{m+(i-1)k}b^m$$

$i = 1$ ise bu terim gider ve a^mb^m kalır. Aksi halde dizgi dilin içinde değildir. Bu örneğin bölmeleri ve genel formül $m \leq 40$ için, bütün xyz bölmeleri üzerinde tam taramayla denendi; hiçbir bölmede xy^2z dilde çıkmadı.

10. Örnek 2: $\{a^nb^lc^{n+l} \mid n, l \geq 0\}$

Teorem. $L = \{a^nb^lc^{n+l} \mid n, l \geq 0\}$ düzenli değildir.

Hoca bu dilin bir öncekine göre biraz daha esnek olduğunu söyledi. Önce birkaç dizgi görelim. İkisi de sıfırsa boş dizgi elde edilir. $n = 1$, $l = 0$ için ac, $n = l = 1$ için $abcc$ elde edilir.

> **Ders sonrası not:** Hoca teoremi okurken önce "non-linear" dedi, kastedilen nonregular yani düzenli olmayandır. Bu sürçme bir yana, "ikisi de sıfırsa boş küme" dedi; doğrusu boş dizgidir (ε).

Başlangıç. Çelişki için L'nin düzenli olduğunu varsayalım. L sonsuz olduğundan pompalama önsavını kullanabiliriz. m kritik uzunluk olsun.

> **Hocanın vurgusu:** Buraya kadarki kısım her ispatta aynıdır.

Desen seçimi. İçinde m geçen ve uzunluğu en az m olan bir desen gerekiyor. Doğal bir seçim şudur:

$$w = a^m b^m c^{2m}$$

Bu dizgi dildedir, çünkü $n = l = m$ için $n + l = 2m$ olur. Uzunluğu $m + m + 2m = 4m$'dir ve $4m \geq m$'dir.

Bölme. $w = xyz$ yazıyoruz. Koşullar $|xy| \leq m$ ve $|y| \geq 1$'dir. Bu iki koşul farklı bölmelerin sayısını azaltır. Bütün bölmelerin çelişki verdiğini göstermemiz gerektiği için bu önemli bir kolaylıktır.

$$w = \underbrace{a \cdots a}_{m}\,\underbrace{b \cdots b}_{m}\,\underbrace{c \cdots c}_{2m}$$

x dizginin başından başlar ve $|xy| \leq m$ olduğundan xy bu a'ların içinde kalır. Demek ki y yalnız a'lardan oluşur:

$$y = a^k, \qquad 1 \leq k \leq m$$

k en az 1'dir, çünkü y'de en az bir eleman olmalıdır. En çok m'dir, bu durumda x boş dizgidir.

> **Ders sonrası not:** Hoca koşulu söylerken "y'nin uzunluğu m'den büyük" dedi, doğrusu $|y| \geq 1$'dir. Bir yerde de "k tane a x'te" dedi, hemen "y'de" diye düzeltti.

Aşağı pompalama. Önsava göre her $i \geq 0$ için $xy^i z$ dilde olmalıdır. Önceki örnekte yukarı pompalamıştık (pump up), yani y'lerin sayısını artırmıştık. Bu kez aşağı pompalayalım (pump down), yani y'lerin sayısını azaltalım. $i = 0$ alalım. y'yi hiç kullanmayınca k tane a eksilir, b'lere ve c'lere dokunulmaz:

$$xy^0 z = xz = a^{m-k} b^m c^{2m}$$

$k \geq 1$ için bu dizgi dilde olmalıdır. Ama dilde c sayısı, a sayısı ile b sayısının toplamına eşit olmak zorundadır. Burada

$$(m - k) + m = 2m - k \neq 2m$$

çünkü k sıfır değildir. Dolayısıyla $a^{m-k} b^m c^{2m} \notin L$ olur. Bu bir çelişkidir.

Sonuç. L'nin düzenli olduğu varsayımı doğru değildir. İspat biter.

Hoca burada da "$k > 1$" diye yazdı. Doğru koşul $k \geq 1$'dir, yukarıdaki eşitsizlik $k = 1$ için de geçerlidir. Bütün xyz bölmeleri $m \leq 40$ için tam taramayla denendi; hepsinde $y = a^k$ çıktı ve xz dilin dışında kaldı.

11. Örnek 3: $\{a^{n!} \mid n \geq 0\}$

Teorem. $L = \{a^{n!} \mid n \geq 0\}$ düzenli değildir.

Başlangıç. Başlangıç yine aynıdır. Çelişki için L'nin düzenli olduğunu varsayalım. L sonsuz olduğundan pompalama önsavını kullanabiliriz. m kritik uzunluk olsun.

Desen seçimi. İçinde m geçen ve uzunluğu en az m olan bir desen gerekiyor. Doğal seçim şudur:

$$w = a^{m!}, \qquad |w| = m! \geq m$$

> **Ders sonrası not:** Hoca "$m!$, m'den büyüktür" dedi. $m = 1$ ve $m = 2$ için $m! = m$'dir. Desen için gereken koşul $|w| \geq m$ olduğu için bu ispatı bozmaz.

Bölme. $w = xyz$ yazıyoruz, $|xy| \leq m$ ve $|y| \geq 1$ koşullarıyla. Bu örnekte dizginin tamamı a'lardan oluşur. Yine çok yardımcı olan $|xy| \leq m$ koşuludur: Dizginin başından bir noktaya kadar olan kısım xy'dir. x boş ya da herhangi bir uzunlukta olabilir, ne olduğu bizi ilgilendirmez. y için ise

$$y = a^k, \qquad 1 \leq k \leq m$$

yazabiliriz. k en az 1'dir, çünkü $|y| \geq 1$'dir. En çok m'dir, çünkü x boş olup xy'nin tamamı y olabilir. Geri kalan kısım z'dir.

Yukarı pompalama. $i = 2$ alalım. Özgün dizgide $xyz = a^{m!}$ idi. Bir y daha, yani bir a^k daha eklenince

$$xy^2z = a^{m!+k}$$

olur. Önsava göre bu dizgi dilde olmalıdır. Bunun anlamı, $m! + k$'nin bir sayının faktöriyeli olması gerektiğidir. Yani

$$m! + k = n!$$

eşitliğini sağlayan bir n bulunmalıdır. Hatırlayalım, k'nin sınırları vardır: en az 1, en fazla m.

Böyle bir n yoktur. Bunu bir eşitsizlik zinciriyle gösterelim. Burada $m > 1$ varsayılır:

$$m! + k \;\leq\; m! + m \;<\; m! \cdot m + m! \;=\; m!\,(m+1) \;=\; (m+1)!$$

Adımlar şöyledir.

1. $k \leq m$ olduğundan $m! + k \leq m! + m$'dir.
2. m yerine $m! \cdot m$ yazmak sayıyı büyütür, çünkü $m > 1$ için $m! > 1$'dir. Bu adımda eşitsizlik kesinleşir.
3. $m!$ ortak çarpan olarak alınınca $m!\,(m+1)$ olur.
4. Bu da $(m+1)!$'in tanımıdır.

Demek ki $m! + k$, $m!$'den sonraki en küçük faktöriyel olan $(m+1)!$'den küçüktür. Bu yüzden hiçbir n için $n!$'e eşit olamaz:

$$m! + k \neq n! \quad \text{(her } n \text{ için)}$$

Bu, "böyle bir n vardır" sonucuyla çelişir. Yani $xy^2z = a^{m!+k}$ dilde değildir.

Sonuç. L'nin düzenli olduğu varsayımı yanlıştır. Böylece pompalama önsavını üç kez uygulamış olduk.

> **Ders sonrası not:** Hoca yalnız üst sınırı, yani $m! + k < (m+1)!$ kısmını gösterdi. İspatın tamamlanması için alt sınır da gerekir: $k \geq 1$ olduğundan $m! + k > m!$'dir. İkisi birlikte $m! < m! + k < (m+1)!$ verir. Ardışık iki faktöriyelin arasında kalan sayı faktöriyel olamaz. Bu zincir $2 \leq m \leq 15$ ve bütün $1 \leq k \leq m$ için hesapla doğrulandı.
>
> **Ders sonrası not:** Hocanın "$m > 1$ varsayımı" gereklidir. $m = 1$, $k = 1$ için $1!+1 = 2 = 2!$ olur, yani çelişki çıkmaz. Varsayım genelliği bozmaz: Bir m kritik uzunluk ise ondan büyük her sayı da kritik uzunluktur, çünkü koşul daha az dizgi için istenir ve $|xy| \leq m$ daha büyük sayı için de geçerlidir. Bu yüzden $m \geq 2$ seçilebilir.
>
> **Ders sonrası not:** Hoca bu örneğin sonlarında birkaç kez sürçtü. "a^{m+k}" dedi, doğrusu $a^{m!+k}$'dir. "$n!+k = n!$" dedi, doğrusu $m!+k = n!$'dir. "m'den sonraki en küçük faktöriyel" dedi, kastedilen $m!$'den sonrakidir.

12. Özet: Düzenlilik ve Düzensizlik Nasıl İspatlanır?

Hoca dersi şu özetle bağladı.

Sonlu dil. L sonluysa düzenlidir, burada sorun yoktur.

Sonsuz dil. L sonsuzsa düzenli de olabilir, düzenli olmayabilir de. Bunu ispatlamak gerekir.

Düzenlilik ispatı. Bir L dilinin düzenli olduğunu, onu kabul eden bir sonlu otomat vererek ya da onu üreten bir düzenli ifade (regular expression) vererek gösterebiliriz. "Bu M makinesi bu dili kabul ediyor, öyleyse bu dil düzenlidir" deriz. Düzenli ifadeler sonlu otomatlara denktir. Düzenli ifadeleri bu derste görmedik. Hoca bunların çok kolay olduğunu, Sipser'da 63-66. sayfalardan okunmasını söyledi.

Tasarlayamamak ispat değildir. Sonlu otomat bulamamanız, dilin düzenli olmadığının kanıtı değildir. Tasarlayabilirseniz dil düzenlidir. Tasarlayamıyorsanız bu, dilin düzensiz olduğunu göstermez.

> **Ders sonrası not:** Hoca bu cümlede "tasarlayamazsanız bu, dilin düzenli olduğunu göstermez" dedi. Kastedilen, dilin düzenli **olmadığını** göstermez.

Düzensizlik ispatı. Bir L dilinin düzenli olmadığını pompalama önsavıyla kanıtlayabiliriz.

Küme kapanma özellikleri. Son olarak, bir dilin düzenli ya da düzensiz olduğunu göstermek için küme kapanma özelliklerini (set closure) de kullanabiliriz. Bunları daha önce konuşmuştuk. Örneğin düzenli diller kesişim altında kapalıdır (closed under intersection): İki düzenli dilin kesişimi de düzenli bir dildir. Bu fikir, bir dilin düzenli olduğunu kanıtlamak için kullanılabilir.

> **Ek bilgi (derste anlatılmadı):** Kapanma özelliği düzensizlik yönünde de çelişkiyle kullanılır. L dili düzenli olsaydı, bir düzenli R diliyle kesişimi $L \cap R$ de düzenli olurdu. Eğer $L \cap R$'nin düzenli olmadığı biliniyorsa, L'nin de düzenli olmadığı sonucu çıkar.

Dersin Toparlaması

Derse, geçen hafta yarım kalan NFA'dan DFA'ya dönüşüm örneğini bitirerek başladık. DFA durumlarını NFA durum kümeleriyle adlandırdık. Her yeni durum için her sembolü sorduk, boş geçişleri hedef kümeye kattık, geçişi olmayan yerler için $\emptyset$ tuzak durumunu açtık ve içinde NFA kabul durumu bulunan kümeleri kabul durumu yaptık. Ardından aynı işi kuvvet kümesi üzerinden genel bir yöntem olarak yazdık.

Sonra düzenli olmayan dillere geçtik. Güvercin yuvası ilkesinden yola çıkarak, uzunluğu durum sayısından az olmayan bir dizginin yürüyüşünde bir durumun tekrar etmesi gerektiğini gördük. Bu tekrar bir döngü demektir ve dizgiyi x, y, z parçalarına böler. Böylece pompalama önsavının üç koşuluna ulaştık: $|xy| \leq m$, $|y| \geq 1$ ve her $i \geq 0$ için $xy^iz \in L$. Önsavı çelişkiyle ispat kalıbı içinde $\{a^nb^n\}$, $\{a^nb^lc^{n+l}\}$ ve $\{a^{n!}\}$ dillerine uyguladık. İlkinde ve üçüncüsünde yukarı, ikincisinde aşağı pompaladık. Bölüm 1'in anlatımı burada bitti. Düzenli ifadeler derste atlandığı için kitaptan okunacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.